University of Central Punjab

Faculty of Information Technology & Computer Sciences

Data Structures, Spring 2026

Assignment 1

Total Marks: 50	**Deadline: 20 - 03 - 2026**	**Section:**
Name: Sharjeel	**Reg. #:** L1F24BSCS0506	

CLO	CLO Statement	Taxonomy Level	PLO
1	Understand the basic knowledge of data structures and algorithms.	C2	2

Instructions: (5 marks will be deducted for not following the instructions)

- You have to submit Assignment 1 via Portal. Do individually.
- Quiz will be taken on the basis of assignment.
- In case of plagiarism, ***straight ZERO*** will be awarded.
- Attempt all questions in sequence. **Attach this title page as a front page of assignment**.
- Assignments will not be accepted on email and late submissions of assignments are strictly prohibited.
- Attach output screen shot of each question.

Answers Clarity: Justify your answer where needed.

Question – 01

arr = {12, 77, 87, 89, 100, 117, 125, 189, 235, 529, 1000}
to Find = 18
arr Size = 11

Iteration	Low	High	Mid	arr[Mid]	Decision
1	0	10	5	117	18 < 117 → High = Mid - 1 = 4
2	0	4	2	87	18 < 87 → High = Mid - 1 = 1
3	0	1	0	12	18 > 12 → Low = Mid + 1 = 1
4	1	1	1	77	18 < 77 → High = Mid - 1 = 0

Result

The number 18 wasn't found in the array

Total Comparisons

4 comparisons

Question – 02

```
TERMINAL                                                                    bash

sharjeel@sharjeel:~/0/zcp/0repos/semester-04/ds/assignments/01$ g++ task-02.cpp -o x && ./x
Enter your first name: sharjeel
Task 0: Initial array
Ascii: 115 104 97 114 106 101 101 108
Char: sharjeel

Enter your second name: ahmad

Task 01: Last name first letter's ASCII at position 2 (index 1) in array
Ascii: 115 97 97 114 106 101 101 108
Char: saarjeel

Task 02: Deleted value at position 3 (index 2)
New size of array: 7
Ascii: 115 97 114 106 101 101 108
Char: sarjeel

Task 03: Changed last element to 100
Ascii: 115 97 114 106 101 101 100
Char: sarjeed
sharjeel@sharjeel:~/0/zcp/0repos/semester-04/ds/assignments/01$

Ln 27, Col 16   Spaces: 4   UTF-8   LF   { } C++   Finish Setup   Linux   Background
```

Question – 03

push()

```
TERMINAL                                                                bash - task-03
sharjeel@sharjeel:~/0/zcp/0repos/semester-04/ds/assignments/01/task-03$ g++ task-03.cpp -o x && ./x
Pushing 'L'
Current Size: 1
Max Size: 5
1. L
----------------------
Pushing '1'
Current Size: 2
Max Size: 5
1. 1
2. L
----------------------
Pushing 'F'
Current Size: 3
Max Size: 5
1. F
2. 1
3. L
----------------------
Pushing '2'
Current Size: 4
Max Size: 5
1. 2
2. F
3. 1
4. L
----------------------
Pushing '4'
Current Size: 5
Max Size: 5
1. 4
2. 2
3. F
4. 1
5. L
----------------------
Ln 7, Col 1   Spaces: 4   UTF-8   LF   { } C++   Finish Setup   Linux   Background
```

```
----------------------
Pushing 'x' will result in stack overflow error.

Stack overflow
Current Size: 5
Max Size: 5
1. 4
2. 2
3. F
4. 1
5. L
----------------------
```

pop()

```
TERMINAL                                                    bash - task-03
sharjeel@sharjeel:~/0/zcp/0repos/semester-04/ds/assignments/01/task-03$ g++ task-03.cpp -o x && ./x
Popping '4'
Current Size: 4
Max Size: 5
1. 2
2. F
3. 1
4. L
---------------------
Popping '2'
Current Size: 3
Max Size: 5
1. F
2. 1
3. L
---------------------
Popping 'F'
Current Size: 2
Max Size: 5
1. 1
2. L
---------------------
Popping '1'
Current Size: 1
Max Size: 5
1. L
---------------------
Popping 'L'
Current Size: 0
Max Size: 5
Stack is empty
---------------------
```

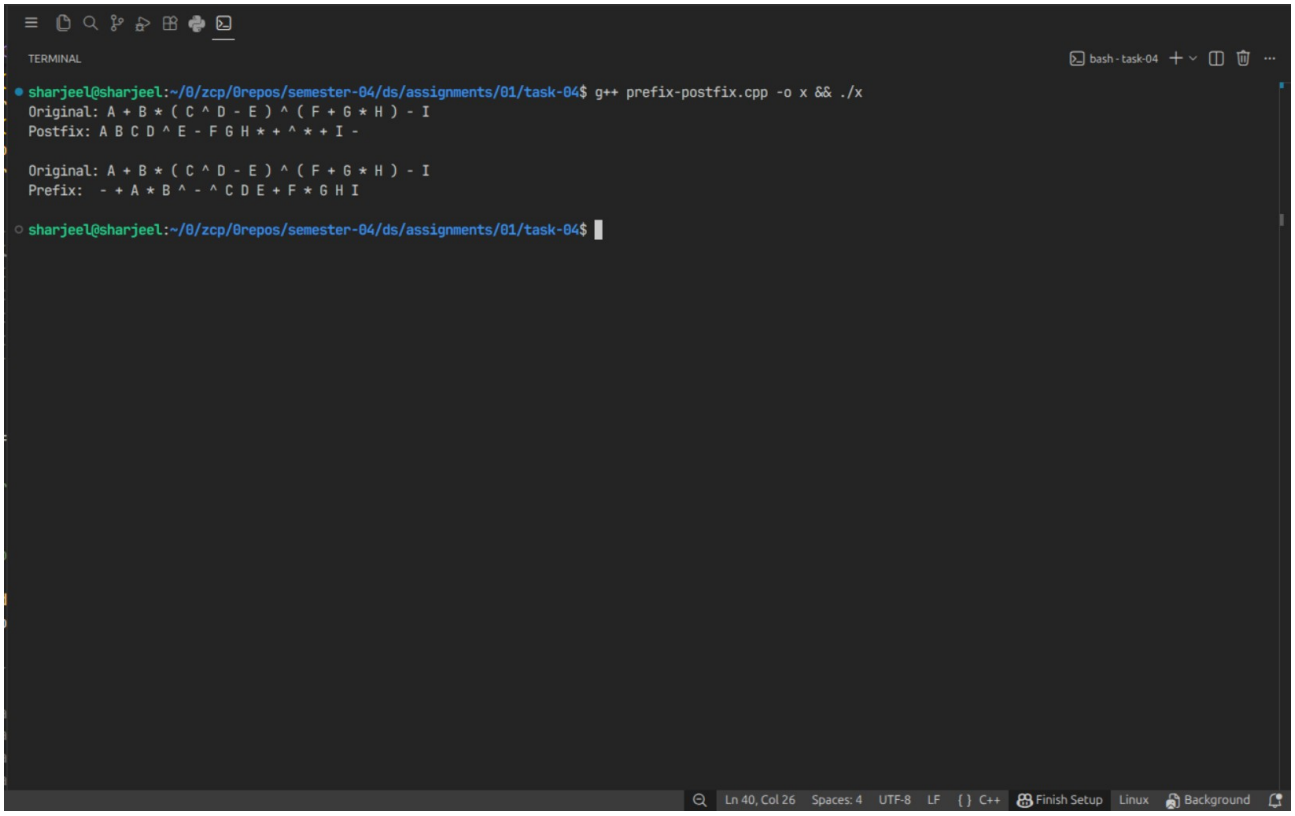
```
-----------------------
Popping will result in stack underflow error.

Stack underflow
Current Size: 0
Max Size: 5
Stack is empty
-----------------------
```

Question – 04

A + B * (C ^ D - E) ^ (F + G * H) - I

It.	T	Stack	Postfix	Prefix
1	A	[]	A	A
2	+	[+]	A	+ A
3	B	[+]	A B	+ A B
4	*	[+, *]	A B	+ A * B
5	(	[+, *, (]	A B	+ A * B
6	C	[+, *, (]	A B C	+ A * B C
7	^	[+, *, (, ^]	A B C	+ A * B ^ C
8	D	[+, *, (, ^]	A B C D	+ A * B ^ C D
9	-	[+, *, (, -]	A B C D ^	+ A * B ^ C D
10	E	[+, *, (, -]	A B C D ^ E	+ A * B - E ^ C D
11	)	[+, *]	A B C D ^ E -	+ A * B - ^ C D E
12	^	[+, *, ^]	A B C D ^ E -	+ A * ^ B - ^ C D E
13	(	[+, *, ^, (]	A B C D ^ E -	+ A * ^ B - ^ C D E
14	F	[+, *, ^, (]	A B C D ^ E - F	+ A * ^ B - ^ C D E F
15	+	[+, *, ^, (, +]	A B C D ^ E - F	+ A * ^ B - ^ C D E + F
16	G	[+, *, ^, (, +]	A B C D ^ E - F G	+ A * ^ B - ^ C D E + F G
17	*	[+, *, ^, (, +, *]	A B C D ^ E - F G	+ A * ^ B - ^ C D E + F * G
18	H	[+, *, ^, (, +, *]	A B C D ^ E - F G H	+ A * ^ B - ^ C D E + F * G H
19	)	[+, *, ^]	A B C D ^ E - F G H * +	+ A * ^ B - ^ C D E + F * G H
20	-	[-]	A B C D ^ E - F G H * + ^ * +	- + A * B ^ - ^ C D E + F * G H
21	I	[-]	A B C D ^ E - F G H * + ^ * + I	- + A * B ^ - ^ C D E + F * G H I
22		[]	A B C D ^ E - F G H * + ^ * + I -	- + A * B ^ - ^ C D E + F * G H I

Output


TERMINAL
bash - task-04
sharjeel@sharjeel:~/0/zcp/0repos/semester-04/ds/assignments/01/task-04$ g++ prefix-postfix.cpp -o x && ./x
Original: A + B * (C ^ D - E) ^ (F + G * H) - I
Postfix: A B C D ^ E - F G H * + ^ * + I -
Original: A + B * (C ^ D - E) ^ (F + G * H) - I
Prefix: - + A * B ^ - ^ C D E + F * G H I
sharjeel@sharjeel:~/0/zcp/0repos/semester-04/ds/assignments/01/task-04$
Ln 40, Col 26
Spaces: 4
UTF-8
LF
{ } C++
Finish Setup
Linux
Background

Question – 05

Task 01 — Time Complexity

Task	Description	TC	Justification
T1	Retrieve student record using index	**O(1)**	Direct index access jumps straight to the memory location
T2	Search student in unsorted list	**O(N)**	Must check every record one by one in worst case
T3	Compare all students with each other	**$O(N^2)$**	Every student is compared against every other student. For N students that is $N\times(N-1)/2$ comparisons
T4	Search sorted list using Binary Search	**$O(\log_2 N)$**	Each step cuts the remaining search space in half, so even 1 million records only needs ~20 comparisons

Task 02 — Efficiency Ranking

Rank	Task	Complexity	N = 1,000,000
1st (Best)	T1 — Index Retrieval	O(1)	1 operation
2nd	T4 — Binary Search	$O(\log_2 N)$	~20 operations
3rd	T2 — Unsorted Search	O(N)	1,000,000 operations
4th (Worst)	T3 — Duplicate Detection	$O(N^2)$	1,000,000,000,000 operations

Task 03 — Optimization Recommendations

T3 (Duplicate Profile Detection) must be optimized At $O(N^2)$, with several million records this becomes tedious. Even powerful servers would take hours or crash entirely. This is completely unusable at that scale.

T2 (Unsorted List Search) should be redesigned Searching one record at a time through millions of unsorted records means the system slows down linearly as the database grows. If students are frequently searched by name or ID, this becomes slow.

Task 4 — Alternative for Least Efficient Task

Least efficient: T3 -> $O(N^2)$ duplicate detection

Recommended alternative: Hash Table

Instead of comparing every student against every other student, insert each student record into a hash table using a unique key (name + roll no). If inserting a record produces a collision on a meaningful field, a duplicate is detected immediately.

Approach	Complexity
Current (brute force compare)	$O(N^2)$
Hash Table duplicate detection	$O(N)$